

Explainable AI for Cybersecurity: Interpreting Machine Learning Decisions in Intrusion Detection Systems

Rajat Kumar[@] and Sadhil Madan[#]

Department of Information Technology,

K. J. Somaiya School of Engineering, Mumbai, India (formerly known as K. J. Somaiya College of Engineering)

@ORCID: 0009-0004-1688-0855 #ORCID: 0009-0001-3944-5459

Abstract—An intrusion alert is useful only if an analyst can assess the evidence behind it. We trained Random Forest (RF) and XGBoost (XGB) for binary attack detection on the published UNSW-NB15 training partition and assessed them on its held-out test partition. We then examined SHAP and LIME explanations separately for correct attack detections, correct benign classifications, false alarms, and missed attacks. At the prespecified 0.5 threshold, RF achieved F1=0.9055, recall=0.9818, FPR=0.2287, and ROC-AUC=0.9840. XGB achieved F1=0.8962, recall=0.9833, FPR=0.2587, and ROC-AUC=0.9840. RF produced 1,110 fewer false alarms, whereas XGB detected 65 more attacks. The models shared five attributes in their global SHAP top tens, although their full rankings correlated only moderately. Across outcome groups, median overlap between the five leading SHAP and LIME features ranged from 0.111 to 0.429. TreeSHAP ran faster; XGB's LIME surrogates showed higher fidelity. The disagreement between explainers makes their outputs useful for examining a decision, but insufficient to establish that an alert is correct or that a feature caused an attack.

Index Terms—cybersecurity, explainable artificial intelligence, intrusion detection system, LIME, SHAP

I. INTRODUCTION

Intrusion detection systems examine network events for signs of misuse, attack, or policy violation. Machine-learning models extend this task beyond fixed signatures by learning statistical patterns from labelled traffic. This flexibility is valuable when attack behaviour varies, but it also creates an operational problem: a model may label a flow as malicious without giving the analyst a reason that can be checked against the traffic context.

An unexplained false positive can waste investigation time or encourage alert fatigue. An unexplained false negative is more serious because harmful activity may pass without review. Accuracy alone hides this difference. A useful evaluation must therefore report recall and false-positive rate, preserve the natural test distribution, and examine why correct and incorrect predictions receive their scores.

An analyst can inspect the same alert in two ways. With SHAP, the model output is expressed relative to a reference output and divided into feature contributions [3]. With LIME, nearby versions of that flow are sent through the classifier to fit a local surrogate [2]. We compare what the methods highlight instead of assuming they will tell the same story. Neither result identifies what caused the network event; each describes a response of the fitted model under particular explanation settings.

Random Forest and XGBoost were trained and tested on the published UNSW-NB15 partitions. We use their held-out predictions to form four case groups: correctly detected attacks, correctly classified benign flows, false alarms, and missed attacks. Detection scores describe the models on the full test set; the case groups let us inspect where SHAP and LIME agree or diverge. Adding an explanation does not itself change a prediction, so we evaluate those two questions separately.

For the selected flows, we record the attributes each method places near the top of its explanation. We also measure how

closely LIME's surrogate follows the classifier locally, whether seeded reruns give similar results, and the time required to produce each explanation. Those observations tell us about the outputs of the tools. Whether a security analyst would understand an alert faster or respond more accurately remains an open question for a user study.

The paper reports detection on the complete official test file, then examines seeded examples from each of the four prediction outcomes. Feature overlap, LIME fidelity, stability, and timing accompany the plots so readers can judge what an explanation does and where it is limited. The saved model pipelines, selected case identifiers, figures, and result files make the reported values traceable to the experiment run.

II. BACKGROUND AND RELATED WORK

A. Explainability Methods

Our analysis moves between two scales. Across the sampled test records, we rank features by their mean absolute SHAP contribution to see which inputs the fitted model uses most often. For selected alerts, we inspect the individual SHAP and LIME outputs to see what drove that particular score. The first view can reveal a recurring dependence on a traffic field; it cannot explain every single alert. The second view is specific to one flow and must be read with its prediction outcome.

LIME explains a prediction by sampling around the selected record, obtaining model outputs for those samples, and fitting an interpretable local model [2]. Its model-agnostic design is useful when the same explanation procedure must be applied to different classifiers. The local approximation can nevertheless become unstable when sampling changes or when perturbations create implausible combinations of network attributes.

The choices made around LIME are part of the experiment rather than minor software settings. Kernel width controls how strongly distant samples influence the surrogate. The number of perturbed samples affects runtime and the precision of the

fitted local model. Because UNSW-NB15 includes related counts, durations, rates, and protocol fields, perturbations may be mathematically valid but unlike feasible flows. The implementation preserved categorical values and interpreted LIME as a local approximation.

For SHAP, a model output is decomposed into a reference value and attributed contributions from the input features [3]. The tree implementation makes this calculation practical for the two ensembles studied here. We use aggregate attributions to inspect recurring model patterns and record-level plots to inspect individual alerts. Neither view is independent of the fitted model or the representation of its inputs. In particular, correlated traffic measurements may share importance differently under different attribution assumptions, so a large contribution should be read in that context.

For a prediction $f(x)$, an additive explanation can be written conceptually as $f(x) = \phi_0 + \sum \phi_i$, where ϕ_0 is the reference output and ϕ_i is the contribution assigned to feature i . Positive and negative contributions are meaningful only relative to the selected class and output scale. This study used raw-output TreeSHAP values and probability-based LIME surrogates, so attribution magnitudes were not compared directly across methods.

B. Explainable Intrusion Detection

Wang et al. presented an early explainable machine-learning framework for IDS using SHAP with one-versus-all and multiclass classifiers on NSL-KDD [4]. The explanations corresponded to known attack characteristics, establishing a practical link between feature attribution and intrusion analysis. The use of an older benchmark and the limited treatment of explanation reliability leave room for evaluation on other traffic representations.

Patil et al. trained decision tree, Random Forest, SVM, and voting models on CICIDS2017 and used LIME to explain individual results [5]. Their study shows that ensemble detection and local explanation can be integrated. It concentrates on LIME and reported classifier performance, so agreement with another explainer and a systematic comparison of false positives and false negatives remain separate questions.

Wei et al.'s xNIDS accounts for input history and feature dependencies when explaining deep intrusion detectors [6]. Its evaluation examines fidelity, sparsity, completeness, and stability, and the authors also explore using explanations to derive defence rules. Those tests motivate our choice to measure explanation properties explicitly. The architecture and sequential input setting, however, differ from the tabular ensembles used in this experiment.

Across CICIDS2017, NSL-KDD, and RoEduNet-SIMARGL2021, Arreche et al. evaluated seven classifiers alongside SHAP and LIME [7]. Their work already establishes the value of comparing explainers across models and datasets. Here we ask a narrower question on UNSW-NB15: what changes when the same evaluation protocol is applied to RF and XGB and explanations are separated by the four prediction outcomes?

Gaspar et al. studied LIME and SHAP with an MLP intrusion detector built from a generated version of the ADFA-LD system-call data [8]. They assessed perturbations and also gathered user feedback. We adopt the principle that an explanation needs evaluation of its own, while recognising that conclusions about system calls and a neural model cannot simply be carried over to network-flow features and tree ensembles.

Mia et al. inspected SHAP visualisations for correct and incorrect classifications on CIC-IoT2023, NF-UQ-NIDS-v2, and HIKARI-2021 [9]. Their error-focused analysis is particularly relevant to our false-alarm and missed-attack groups. Our comparison adds LIME on the same selected records and reports measured agreement, stability, and computation time rather than judging plots alone.

Hermosilla et al. compared SHAP and LIME for XGBoost and TabNet on UNSW-NB15, including stability and fidelity considerations in a forensic setting [10]. This is the closest prior configuration and prevents a broad novelty claim based only on the dataset and explainers. The present study instead pairs XGBoost with Random Forest and organises the comparison around all four confusion-matrix outcomes.

Mohale and Obagbuwa's systematic review found continued problems in standardisation, scalability, and real-time use of explanations in IDS [11]. Taken together, the literature shows that explanations should be evaluated as outputs with their own limitations. It also shows that explanation plots alone do not establish human understanding or operational trust.

The studies also use different meanings of explanation quality. Some judge whether a plot appears consistent with known attack characteristics. Others perturb important features, compare a surrogate with the original model, test stability, or ask participants to rate usefulness. These measures are complementary rather than interchangeable. The present study therefore uses several measurable properties and labels each one precisely. It does not collapse fidelity, stability, agreement, sparsity, runtime, and human trust into one interpretability score.

Table I summarises the relationship between these studies and the controlled replication and extension reported here.

Table I. Selected literature and its relevance to the present study

Study	Method	Dataset	Main finding	Limitation or gap
Wang et al. [4]	SHAP with neural IDS	NSL-KDD	Local and global attributions align with attack characteristics	Older dataset; limited explanation validation
Patil et al. [5]	Ensemble IDS with LIME	CICIDS2017	Local explanations accompany an ensemble classifier	Single explainer; limited error-stratified analysis
Wei et al. [6]	xNIDS	Four DL-NIDS settings	Measures fidelity, sparsity, completeness, and stability	Deep sequential setting differs from tabular tree models
Arreche et al. [7]	Seven models; SHAP and LIME	CICIDS2017, NSL-KDD, RoEduNet	Broad model and explainer comparison	Does not centre on UNSW-NB15 error outcomes
Gaspar et al. [8]	MLP; LIME and SHAP	Generated ADFA-LD	Perturbation and user feedback evaluate explanations	System-call data; one model family
Mia et al. [9]	SHAP error-group plots	Three traffic datasets	Diagnoses FP and FN patterns	No LIME comparison; visual similarity depends on analyst judgement
Hermosilla et al. [10]	XGBoost, TabNet; SHAP and LIME	UNSW-NB15	Compares fidelity and stability for forensic use	Different model pairing; broader outcome-stratified replication is useful
Mohale and Obagbuwa [11]	Systematic review	Twenty studies	Finds standardisation and scalability gaps	Review evidence does not replace controlled experiments

III. PROBLEM STATEMENT AND RESEARCH GAP

High-performing IDS classifiers can still be difficult to audit. A model comparison based only on accuracy cannot show whether one system misses more attacks, creates more false alarms, or relies on features that are unstable under a different explanation sample. Existing research has already applied SHAP and LIME to IDS, compared multiple models, and studied misclassifications [7]–[10]. The research gap is therefore specific rather than absolute.

We ran RF and XGB through the same UNSW-NB15 evaluation and kept detection scores alongside explanation measurements. On records selected from each confusion-matrix outcome, we compared the two explanation methods and recorded their feature agreement, fidelity, stability, and runtime. The point is to test how these methods behave in a controlled IDS comparison. Earlier work has already introduced the methods themselves, so we make no claim to a new XAI algorithm.

Both models start from the same predictor set and published data partitions. Their configurations were selected within the training file before either model was scored on the official test file. We kept the test distribution intact when calculating precision and false-positive rate. For explanation rankings, one-hot contributions were combined back into the network attributes from which they came. These choices make the model and explanation comparisons easier to interpret without changing the underlying cases.

False alarms and missed attacks deserve inspection for different reasons. For a benign flow flagged as hostile, we ask which attributes drove the alert and whether they also appear in correctly detected attacks. For a missed attack, we inspect the evidence behind the benign score and compare it with the other model's treatment of that flow. Such observations can point to features, thresholds, or data quality issues worth investigating in a later experiment; they do not repair the errors in this run.

IV. RESEARCH QUESTIONS AND OBJECTIVES

A. Research Questions

RQ1: How do Random Forest and XGBoost compare in attack recall, false-positive rate, F1-score, ROC-AUC, and PR-AUC under identical data partitions?

RQ2: Which original network attributes most strongly influence each model, and how much do their global feature rankings overlap?

RQ3: How do SHAP and LIME explanations differ among true positives, true negatives, false positives, and false negatives?

RQ4: How repeatable are the explanations under controlled reruns, and what computation time do they require?

RQ5: Which diagnostic observations can be made from error explanations, and which claims about analyst trust remain untested without a user study?

B. Objectives

- Build a leakage-controlled preprocessing and training pipeline using the official UNSW-NB15 partitions.
- Compare Random Forest and XGBoost using the same test records and clearly defined positive class.
- Generate global and local explanations with network attributes presented in their original semantic groups.
- Analyse correct predictions, false alarms, and missed attacks, then measure explanation agreement, sensitivity, fidelity, and time.

V. METHODOLOGY AND IMPLEMENTATION

We first checked the published dataset schema and partition roles, then fitted preprocessing and model-selection procedures using training data. After refitting the selected models, we scored the untouched test partition and grouped its records by prediction outcome. SHAP and LIME were applied to those stored predictions. Scripts and saved outputs record the fitted pipelines, case identifiers, figures, result tables, random seeds, and software versions.

The predictor matrix contained 42 attributes. The identifier, binary label, and attack-category label were excluded; the

attack category was retained only for error analysis. Protocol, service, and state were one-hot encoded with unknown-category handling, while numeric features were passed to the tree models without scaling. No test labels informed preprocessing or fitting.

Figure 1 summarises the implemented workflow from the dataset audit to the comparison of detection and explanation results.

Fig. 1. Implemented experimental and explanation workflow

Stage	Output
1. Audit and split	Verified schema, counts, labels, and untouched test partition
2. Preprocess	Training-fitted cleaning, encoding, and feature map
3. Train	Tuned Random Forest and XGBoost pipelines
4. Evaluate	Probabilities, confusion outcomes, and detection metrics
5. Explain	SHAP and LIME outputs for global and outcome-group samples
6. Compare	Agreement, stability, fidelity, runtime, and error interpretation

A. Dataset Description

The UNSW-NB15 authors created the dataset in the UNSW Canberra Cyber Range Lab with IXIA PerfectStorm traffic that combines normal activity and simulated attacks [1]. The full release has 2,540,044 records and 49 attributes, including its class label. We use the published files containing 175,341 training records and 82,332 test records. The attack labels cover Analysis, Backdoors, DoS, Exploits, Fuzzers, Generic, Reconnaissance, Shellcode, and Worms [1].

We treat any attack as the positive outcome and reserve the attack-category field for examining errors after prediction. That field does not enter model fitting. Using the supplied test file gives a repeatable comparison, but the age and laboratory setting of UNSW-NB15 limit how far the observed rates can be extended to a live network.

A UNSW-NB15 record describes a flow through protocol, service, state, duration, byte totals, and packet counts. Other fields represent content, timing, and activity involving related hosts, services, or ports [1]. This variety gives analysts several ways to interpret an attribution. It also creates dependence among variables: for example, changing duration or packet count without a consistent change in rate can produce a sampled flow that is numerically possible yet operationally implausible.

The overall binary class totals do not describe the size of every attack category. Consequently, recall calculated over all attacks can conceal a larger miss rate within a smaller category. We report the relevant counts alongside category rates, and avoid strong conclusions where a category has few examples.

Table II records the dataset partitions and the role assigned to each field group in the experiment.

Table II. UNSW-NB15 study profile

Property	Implemented use
Training records	175,341; preprocessing, selection, and fitting
Test records	82,332; final evaluation only
Prediction task	Benign versus attack
Attack category	Evaluation metadata for missed-attack analysis
Feature types	Numeric and categorical network-flow attributes

B. Data Preprocessing and Feature Selection

The supplied files contained 175,341 training rows and 82,332 test rows with the expected 45 columns. The training labels comprised 119,341 attacks and 56,000 benign flows; the test labels comprised 45,332 attacks and 37,000 benign flows. The identifier, label, and attack category did not enter either predictor.

We fitted the one-hot encoder for protocol, service, and state on training records only and configured it to tolerate categories absent from that fit. The other 39 numeric predictors entered the tree models without standardisation. Both saved pipelines include the fitted transformation, ensuring that their test inputs follow the same mapping.

The experiment retained all 42 predictors and the original class distribution. No resampling, test-set balancing, or test-driven feature selection was performed. This full-feature design keeps the comparison focused on the classifiers and explainers and prevents a feature-selection loop from contaminating the final evaluation.

C. Machine Learning Models

RF combines trees fitted to bootstrap samples with random subsets of predictors [12]. This averaging can make its output less sensitive to any one tree. XGB instead adds trees in sequence, with each stage addressing errors left by the current ensemble; regularisation and subsampling constrain its fit [13]. Both models yield scores suitable for thresholding and support tree-based SHAP, allowing us to compare their explanations under a shared procedure.

A fixed 20% stratified validation subset of the training file compared three configurations per model using seed 42. The selected Random Forest used 220 trees, unrestricted depth, minimum leaf size two, square-root feature sampling, and balanced bootstrap weights. The selected XGBoost model used 260 trees, depth six, learning rate 0.08, and row and column subsampling of 0.85.

Both final pipelines were refitted on all training records and evaluated at the declared 0.5 decision threshold. Continuous scores were retained for ROC-AUC and PR-AUC. No probability calibration was added, so probability magnitudes are interpreted as model scores rather than calibrated event frequencies.

D. Explainable AI Techniques

Approximate TreeSHAP produced raw-output attributions for each tree ensemble. Contributions from one-hot columns were summed back to their original attributes. Global rankings

used a label-stratified sample of 1,000 test records, containing 500 benign and 500 attack flows.

LIME called each complete preprocessing-and-model pipeline. Categorical perturbations were constrained to observed protocol, service, and state values. Each explanation used 1,500 perturbations. Its local surrogate R^2 was retained as fidelity evidence rather than assuming the surrogate matched the classifier.

For each model, we selected 15 records with a fixed seed from each of the TP, TN, FP, and FN groups. We compared the sets of five leading SHAP and LIME features using Jaccard overlap. A separate stability check reran LIME with three seeds on five records per group. We also repeated TreeSHAP with identical inputs and settings. Because that tree-path calculation is deterministic, the latter check establishes repeatability under fixed conditions, not robustness to a new background distribution.

E. Experimental Setup

Experiments ran on Windows 11 with Python 3.12.14, scikit-learn 1.9.1, XGBoost 3.4.1, SHAP 0.52.0, and LIME 0.2.0.1; the random seed was 42. Model selection used only training data. We chose the decision threshold of 0.5 before calculating scores on the official test file.

Case selection operated on saved predictions, making each reported explanation traceable to a test record. We sampled equally from the four outcomes to support comparisons, so that sample does not represent their prevalence in ordinary traffic. Timings came from the implemented batch SHAP operation and separate LIME calls on one machine; they describe this workflow rather than a portable hardware benchmark.

F. Evaluation Metrics

We report accuracy, precision, recall, F1, false-positive rate, ROC-AUC, and PR-AUC for detection. With attacks designated positive, precision is $TP/(TP+FP)$, recall is $TP/(TP+FN)$, and false-positive rate is $FP/(FP+TN)$; F1 combines precision and recall through their harmonic mean. We also provide confusion counts and attack-category misses so that readers can see the denominators behind the summary rates.

For explanations, we calculated top-five Jaccard overlap, sign agreement for features shared by both methods, the R^2 of each LIME surrogate, stability over seeded LIME runs, repeatability of fixed-input TreeSHAP calls, and median processing time. These measures describe different properties of the displayed account. Even a stable account that agrees across methods is still an account of model behaviour, not proof of network causation.

VI. RESULTS AND ANALYSIS

Table III. Test performance on 82,332 records at threshold 0.5

Model	Accuracy	Prec.	Recall	F1	FPR	ROC-AUC	PR-AUC
Random Forest	0.8872	0.8403	0.9818	0.9055	0.2287	0.9840	0.9880
XGBoost	0.8745	0.8232	0.9833	0.8962	0.2587	0.9840	0.9884

Table III gives the test results at the fixed threshold. RF yielded 28,538 TN, 8,462 FP, 823 FN, and 44,509 TP; XGB yielded 27,428 TN, 9,572 FP, 758 FN, and 44,574 TP. Thus XGB found 65 attacks that RF missed, at the cost of 1,110 additional false alarms. RF had higher accuracy, precision, and F1. ROC-AUC was nearly the same, while XGB held a slight PR-AUC advantage. The ROC, precision-recall, and threshold-specific views appear in Figure 2.

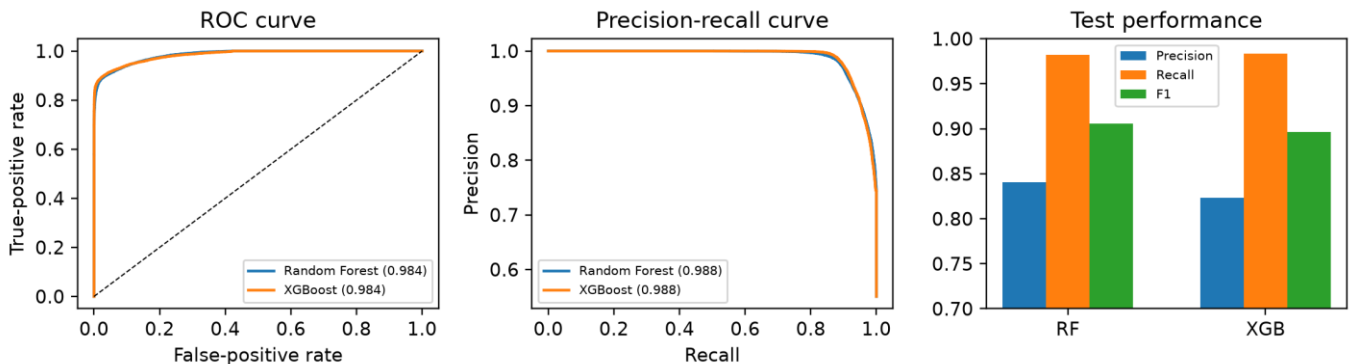


Fig. 2. ROC, precision-recall, and fixed-threshold performance on the official test split

A. Explainability Analysis

Figure 3 shows mean absolute SHAP attributions for the 1,000 sampled test records. The leading attribute was sttl for both models. The next RF attributes were ct_state_ttl, sload, state, and ct_srv_dst; XGB instead placed proto, service, dbytes, and response_body_len near the top. Their top-ten lists shared sttl, service, dbytes, dttl, and ct_srv_dst. That gives a Jaccard score of 0.333, while Spearman correlation over the full rankings was 0.483. The models therefore overlap on some influential fields without assigning the same overall ordering.

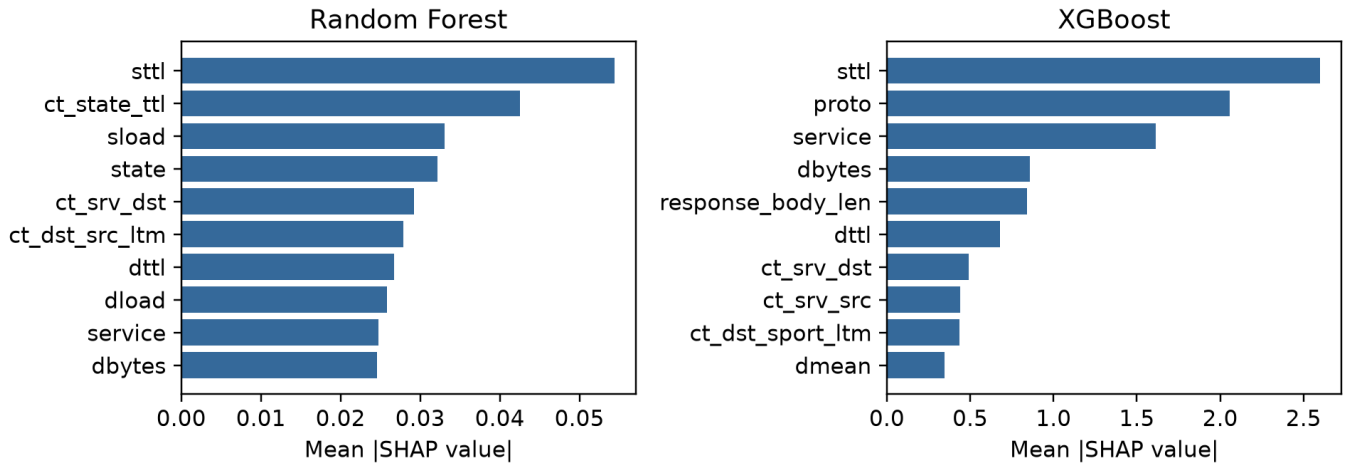


Fig. 3. Global mean absolute approximate TreeSHAP values on 1,000 stratified test records

Table IV. Outcome-stratified local explanation measurements (median values)

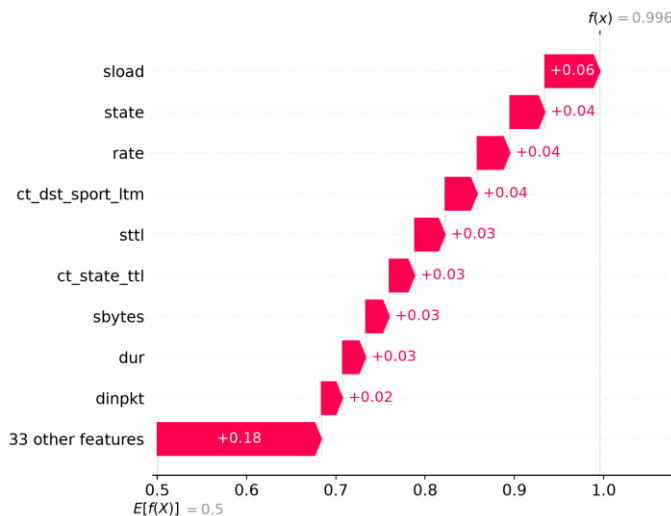
Model	Outcome	n	Top-5 Jacc.	LIME R ²	LIME stab.	SHAP ms	LIME ms
RF	TP	15	0.250	0.637	0.667	3.99	105.44
RF	TN	15	0.429	0.632	0.667	3.99	98.13
RF	FP	15	0.111	0.699	0.667	3.99	99.78
RF	FN	15	0.111	0.722	0.667	3.99	101.76
XGB	TP	15	0.250	0.801	0.429	1.16	61.24
XGB	TN	15	0.250	0.787	0.429	1.16	65.80
XGB	FP	15	0.111	0.776	0.429	1.16	55.32
XGB	FN	15	0.250	0.779	0.429	1.16	56.63

In Table IV, the median overlap of the top-five feature sets was low across outcomes. It reached 0.111 for FP in both models and for RF's FN group. Where the methods selected the same leading feature, median sign agreement was 1.0 for TP and TN but 0.5 for FN. LIME's local fits were stronger for XGB (median R² 0.776–0.801) than for RF (0.632–0.722). Seeded LIME stability was 0.429 for XGB and 0.667 for RF. Identical TreeSHAP calls returned the same values, although that result cannot establish stability under a different background or attribution rule.

B. Representative Local Explanation

Figure 4 compares real explanations for the same Random Forest true-positive prediction: test row 4,254, with an attack score of 0.9959. The SHAP waterfall highlighted sload, state, rate, ct_dst_sport_ltm, and sttl. LIME instead emphasized proto, sttl, ct_state_ttl, dttl, and state, with a local surrogate R² of 0.620. Only sttl and state appeared in both top-five sets, showing why one explanation method should not be treated as the single correct account of a prediction.

Random Forest local SHAP explanation: test row 4254, attack score 0.996



LIME explanation for the same prediction: test row 4254, attack score 0.996



Fig. 4. Local explanations for the same Random Forest true-positive prediction: (a) SHAP waterfall and (b) LIME feature contributions

C. False Positive and False Negative Analysis

A false positive is a benign flow incorrectly labelled as an attack. Operationally, it creates an unnecessary alert and consumes analyst time; a sustained high false-positive rate can produce alert fatigue and delay investigation of real attacks. A false negative is an attack incorrectly labelled as benign. It is more dangerous because malicious traffic may pass without review. XAI does not fix either error automatically, but it helps an analyst inspect which observed features influenced the model's decision.

The error distribution exposes the model trade-off. Random Forest misclassified 22.87% of benign test flows as attacks, compared with 25.87% for XGBoost. Most missed attacks were Fuzzers: 759 of 6,062 for Random Forest (12.52%) and 636 of 6,062 for XGBoost (10.49%). Random Forest had 56 missed Exploits and only eight missed attacks across the remaining reported categories; XGBoost had 63 missed Exploits, 21 DoS, 19 Analysis, six Backdoors, seven Generic, four Shellcode, and two Reconnaissance cases. No Worms were missed by either model in this run.

Errors expose a particular limit of feature attribution. SHAP assigns contributions within the fitted ensemble on its raw-output scale, whereas LIME fits a local approximation to probability outputs generated from perturbed records. Their low overlap on errors should therefore be shown to an analyst as a difference between methods. Neither attribution identifies the physical cause of an attack, and neither alone settles why a particular prediction failed.

VII. DISCUSSION

The results operate on two levels. Random Forest and XGBoost achieved recall values of 0.9818 and 0.9833, respectively, but their FPR values remained substantial at 0.2287 and 0.2587. Random Forest gives the better fixed-threshold balance; XGBoost gives the highest recall by a small margin. Explanation metrics show that the account presented to an analyst changes with the explainer, especially for errors.

Post-hoc explanation does not change a fixed model's predictions. Adding SHAP or LIME therefore cannot directly improve accuracy, recall, or false-positive rate. Explanations may support later model redesign, feature review, or threshold adjustment, but each change requires validation on data that did not inform the change. Once test cases guide redesign, those cases are no longer an untouched final evaluation set.

Claims about trust and practical usability require evidence from security analysts. The present benchmark can demonstrate feature visibility, explanation consistency, and processing cost. It cannot demonstrate faster triage, better analyst decisions, or greater trust without a user study. This boundary is important because readable plots may still omit dependencies or reflect artefacts in the data.

Operational preference determines the model choice. Random Forest reduced false alarms by 1,110 relative to XGBoost, which matters when benign volume is high. XGBoost detected 65 additional attacks and missed fewer Fuzzers, which may matter when missed activity carries the dominant cost. Neither fixed threshold is deployment-ready without site-specific calibration.

In this implementation, approximate TreeSHAP took about 1.16–3.99 ms per sampled record and LIME about 55–105 ms. The difference favours SHAP when many alerts must be reviewed, with LIME reserved for cases where an additional local account would be informative. For those cases, the disagreement itself should remain visible to the reviewer rather than being resolved by selecting whichever plot looks more convincing.

VIII. LIMITATIONS

The published UNSW-NB15 split offers a fixed benchmark, but its laboratory traffic cannot establish performance on contemporary operational networks or encrypted applications. We did not measure transfer across sites or time periods. Combining all attacks into one positive class also masks category differences, and categories with very few errors cannot support stable subgroup estimates.

Our explanation findings are conditional on the encoded features, the fitted models, and the explainer settings. Correlation among attributes, aggregation of one-hot columns, approximate TreeSHAP, and LIME's sampling procedure may all change the attribution pattern. Local comparison used 15 records per outcome and the global analysis used 1,000 records. We did not vary SHAP backgrounds, calculate confidence intervals, or conduct an analyst study. The observed plots and metrics therefore cannot establish how useful the explanations would be during real triage.

IX. FUTURE SCOPE

A next experiment could distinguish attack categories instead of combining them, then evaluate the resulting models across time periods and on another traffic collection. Counterfactual examples would add a different question to the attribution analysis: which feasible changes to a record would change its classification? An analyst study could test whether showing explanations improves decisions or review time and whether method disagreement changes confidence. It would also be useful to profile explanations only for selected high-risk alerts.

X. CONCLUSION

On the official UNSW-NB15 test partition, RF gave the stronger fixed-threshold F1 (0.9055) and lower false-positive rate (0.2287), whereas XGB recalled slightly more attacks (0.9833). Their global SHAP rankings shared several leading traffic attributes but diverged across the full feature list. At the record level, SHAP and LIME often selected different leading evidence, especially for false alarms, and LIME took longer to compute. These explanations did not alter either classifier's

predictions. They provided a way to inspect the models' decisions, subject to the dependence on explainer choice and the limits of the benchmark.

REFERENCES

- [1] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)," in 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.
- [2] M. T. Ribeiro, S. Singh, and C. Guestrin, "Why should I trust you?: Explaining the predictions of any classifier," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 1135–1144, doi: 10.1145/2939672.2939778.
- [3] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in Advances in Neural Information Processing Systems, vol. 30, 2017, pp. 4765–4774.
- [4] M. Wang, K. Zheng, Y. Yang, and X. Wang, "An explainable machine learning framework for intrusion detection systems," IEEE Access, vol. 8, pp. 73127–73141, 2020, doi: 10.1109/ACCESS.2020.2988359.
- [5] S. Patil et al., "Explainable artificial intelligence for intrusion detection system," Electronics, vol. 11, no. 19, Art. no. 3079, 2022, doi: 10.3390/electronics11193079.
- [6] F. Wei, H. Li, Z. Zhao, and H. Hu, "xNIDS: Explaining deep learning-based network intrusion detection systems for active intrusion responses," in 32nd USENIX Security Symposium, 2023, pp. 4337–4354.
- [7] O. Arreche, T. Guntur, and M. Abdallah, "XAI-IDS: Toward proposing an explainable artificial intelligence framework for enhancing network intrusion detection systems," Applied Sciences, vol. 14, no. 10, Art. no. 4170, 2024, doi: 10.3390/app14104170.
- [8] D. Gaspar, P. Silva, and C. Silva, "Explainable AI for intrusion detection systems: LIME and SHAP applicability on multi-layer perceptron," IEEE Access, vol. 12, pp. 30164–30175, 2024, doi: 10.1109/ACCESS.2024.3368377.
- [9] M. Mia, M. M. A. Pritom, T. Islam, and K. Hasan, "Visually analyze SHAP plots to diagnose misclassifications in ML-based intrusion detection," in 2024 IEEE International Conference on Data Mining Workshops, 2024, pp. 632–641, doi: 10.1109/ICDMW65004.2024.00088.
- [10] P. Hermosilla, S. Berrios, and H. Allende-Cid, "Explainable AI for forensic analysis: A comparative study of SHAP and LIME in intrusion detection models," Applied Sciences, vol. 15, no. 13, Art. no. 7329, 2025, doi: 10.3390/app15137329.
- [11] V. Z. Mohale and I. C. Obagbuwa, "A systematic review on the integration of explainable artificial intelligence in intrusion detection systems to enhancing transparency and interpretability in cybersecurity," Frontiers in Artificial Intelligence, vol. 8, Art. no. 1526221, 2025, doi: 10.3389/frai.2025.1526221.
- [12] L. Breiman, "Random forests," Machine Learning, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
- [13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.